App Dev II Project Report

Placement Portal Application

IIT Madras — Online Degree Programme

1. Student Details

Name	Anaypal Singh
Roll Number	23F3000736
Email	23f3000736@ds.study.iitm.ac.in

---

2. Project Details

Project Title

Placement Portal Application

Problem Statement

College placement processes are often fragmented and manual, making it difficult for students to discover opportunities, for companies to manage drives efficiently, and for administrators to oversee the end-to-end workflow. This project aims to build a comprehensive, role-based web application that centralises placement activities — enabling students to browse and apply for drives, companies to post and manage job listings, and admins to approve, monitor, and report on all activities through a unified dashboard.

Approach

The application follows a decoupled full-stack architecture. The backend is built with Flask using the Blueprint pattern for modular routing, with SQLite as the database accessed via SQLAlchemy ORM. Role-based access control (RBAC) is enforced for three user types: Admin, Company, and Student. The frontend is a single-page application built with Vue.js 3 and served as static files, communicating with the backend exclusively through RESTful APIs. Background tasks (email reminders, monthly reports, CSV exports) are handled asynchronously using Celery with a Redis broker.

---

3. AI / LLM Declaration

I used GitHub Copilot / ChatGPT to assist in improving the UI and frontend of the application. AI/LLM usage is around 10–20%, limited to code suggestions and documentation formatting. All final implementation logic, debugging, and integration were done manually.

---

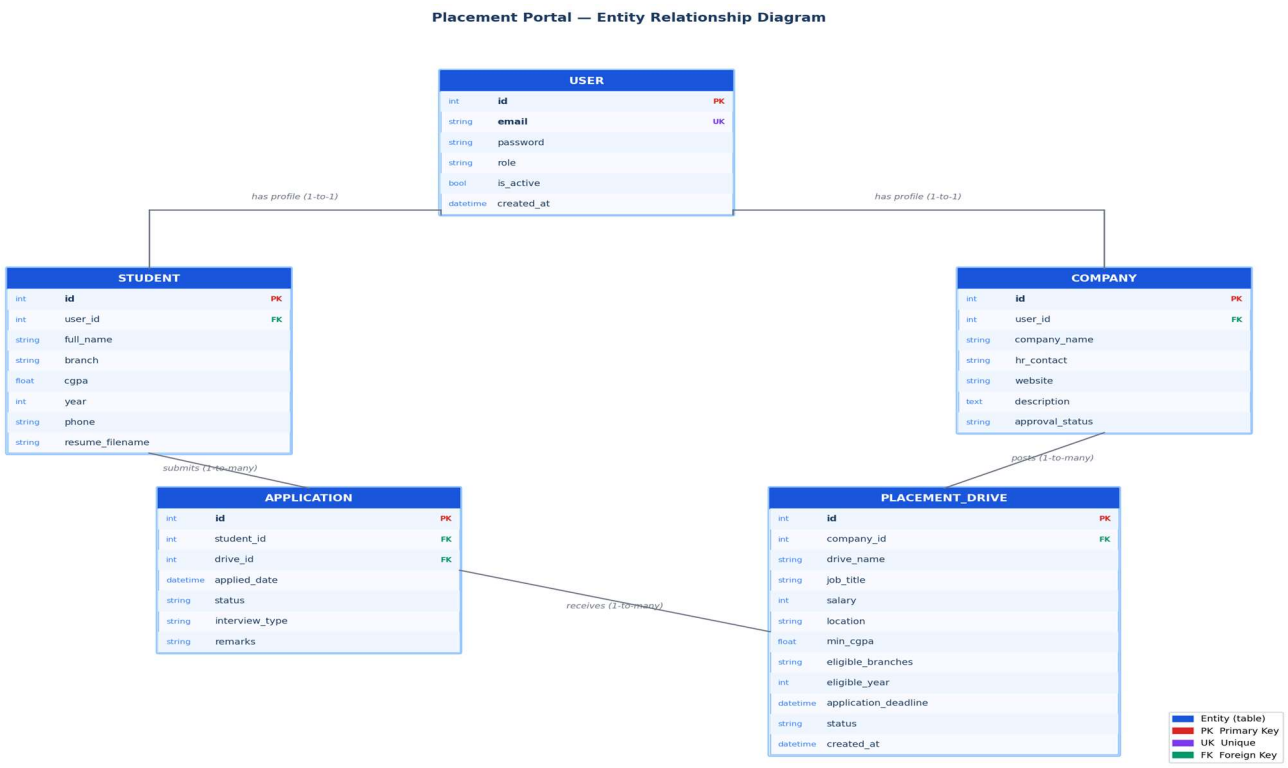
4. Technologies and Frameworks Used

Technology / Library	Purpose
Flask	Core backend web framework for building RESTful APIs
SQLAlchemy	ORM for database modelling and query operations
Flask-Login	User authentication and session management
Flask-CORS	Cross-Origin Resource Sharing support for API access
Flask-Mail / SMTP	Email notifications for placement reminders and reports
Celery	Asynchronous task queue for background jobs (emails, CSV export)
Redis	Message broker for Celery task scheduling
SQLite	Lightweight relational database for data storage
Vue.js 3	Progressive JavaScript framework for reactive SPA frontend
Werkzeug	Password hashing and security utilities

---

5. Database Schema / ER Diagram

The application uses five core tables managed via SQLAlchemy ORM. The diagram below shows the entities, their fields, and relationships.



Tables

1. **user — Stores all users (Admin, Company, Student)**
    - id (PK), email (UK), password, role, is_active, created_at
    - role field distinguishes user type: 'admin' | 'company' | 'student'

2. **student — Extended profile for student users**
    - id (PK), user_id (FK → user), full_name, branch, cgpa, year, phone, resume_filename

3. **company — Extended profile for company users**
    - id (PK), user_id (FK → user), company_name, hr_contact, website, description, approval_status

4. **placement_drive — Job postings created by approved companies**
    - id (PK), company_id (FK → company), drive_name, job_title, job_description, salary, location
    - min_cgpa, eligible_branches, eligible_year, application_deadline, interview_date, status, created_at

5. **application — Tracks which student applied to which drive**
    - id (PK), student_id (FK → student), drive_id (FK → placement_drive)
    - applied_date, status, interview_type, remarks

- UniqueConstraint on (student_id, drive_id) prevents duplicate applications

Relationships

- User → Student / Company (One-to-One via FK)
- Company → PlacementDrive (One-to-Many)
- Student → Application (One-to-Many)
- PlacementDrive → Application (One-to-Many)

---

6. API Resource Endpoints

Authentication APIs (/api/auth)

Endpoint	Method	Description
/api/auth/register/student	**POST**	Register a new student account
/api/auth/register/company	**POST**	Register a new company account
/api/auth/login	**POST**	Authenticate user and create session
/api/auth/logout	**POST**	End current user session
/api/auth/me	**GET**	Get current logged-in user details

Student APIs (/api/student)

Endpoint	Method	Description
/api/student/drives	**GET**	Fetch all approved placement drives (with eligibility check)
/api/student/drives/<id>/apply	**POST**	Apply to a placement drive
/api/student/applications	**GET**	View all applications submitted by the student
/api/student/applications/<id>/status	**GET**	Get status of a specific application
/api/student/profile	**GET/PUT**	View and update student profile
/api/student/resume	**POST**	Upload resume (PDF/DOCX)
/api/student/export/csv	**GET**	Export application history as CSV

Company APIs (/api/company)

Endpoint	Method	Description
/api/company/dashboard	**GET**	Company dashboard with drives summary
/api/company/drives	**POST**	Create a new placement drive

Endpoint	Method	Description
/api/company/drives	**GET**	List all drives posted by this company
/api/company/drives/<id>	**PUT**	Update placement drive details
/api/company/drives/<id>/applicants	**GET**	View all applicants for a drive
/api/company/applicants/<id>/status	**PUT**	Update applicant status (shortlist/select/reject)

Admin APIs (/api/admin)

Endpoint	Method	Description
/api/admin/dashboard	**GET**	System-wide statistics dashboard
/api/admin/companies	**GET**	List all registered companies
/api/admin/companies/<id>/status	**PUT**	Approve / reject a company registration
/api/admin/companies/<id>/blacklist	**PUT**	Blacklist or activate a company account
/api/admin/drives	**GET**	View all placement drives
/api/admin/drives/<id>/status	**PUT**	Approve or reject a placement drive
/api/admin/students	**GET**	List all students
/api/admin/students/<id>/blacklist	**PUT**	Blacklist or activate a student account

---

7. Architecture and Features

Architecture Overview

The project follows a decoupled two-tier architecture:

- Backend (Flask): Modular Blueprint structure
  - run.py — Application entry point
  - app/__init__.py — App factory (create_app), DB init, Blueprint registration
  - app/models.py — SQLAlchemy database models
  - app/auth.py — Authentication Blueprint (/api/auth)
  - app/admin.py — Admin Blueprint (/api/admin)
  - app/company.py — Company Blueprint (/api/company)
  - app/student.py — Student Blueprint (/api/student)
  - app/tasks.py — Celery background tasks (reminders, reports, CSV export)
  - app/cache.py — Caching utilities
  - config.py — Application configuration (DB, mail, Redis, Celery)

- Frontend (Vue.js 3): Single-page application
  - frontend/index.html — Main entry HTML

- src/main.js — Vue app bootstrap
- src/router.js — Client-side routing
- src/components/Login.js — Login/register view
- src/components/StudentDash.js — Student dashboard
- src/components/CompanyDash.js — Company dashboard
- src/components/AdminDash.js — Admin dashboard

Core Features Implemented

User Management

- Student and company self-registration with email + password
- Secure password hashing via Werkzeug
- Session-based authentication with Flask-Login
- Role-based access control: Admin / Company / Student
- Admin-created default account seeded at startup

Student Module

- Browse all approved placement drives with search/filter
- Eligibility check per drive (CGPA, branch, year) before applying
- Apply to drives and track application status in real time
- Upload and manage resume (PDF / DOCX)
- Export complete application history as CSV

Company Module

- Register company and await admin approval before accessing features
- Create placement drives with job details, eligibility criteria, and deadlines
- View applicant list per drive and update status (Applied / Shortlisted / Selected / Rejected)
- Company dashboard with drives summary and applicant counts

Admin Module

- System-wide dashboard: total students, companies, drives, pending approvals, placements
- Approve or reject company registrations and placement drives
- Blacklist / activate student or company accounts
- Receive automated monthly placement activity reports via email

Background Tasks (Celery + Redis)

- Daily reminder emails to students about upcoming drive deadlines (3-day window)
- Monthly placement summary report emailed to the admin on the 1st of each month
- Asynchronous CSV export for student application history
- Heartbeat task for scheduler health monitoring

8. Video Presentation

Drive Link:
https://drive.google.com/file/d/179IGmWPYRxBkB7JZ2_muvDMFZEQ4_aR0/view?usp=drive_link

9. Installation and Setup

Prerequisites

- Python 3.8 or higher
- pip (Python package manager)
- Redis server running locally on port 6379

Installation Steps

1. Extract the project ZIP to your desired location.
2. Open terminal and navigate to the backend/ directory.
3. Create a virtual environment:
   ```
   python -m venv env
   ```
4. Activate the virtual environment:
   ```
   env\Scripts\activate (Windows) | source env/bin/activate (Linux/Mac)
   ```
5. Install dependencies:
   ```
   pip install -r requirements.txt
   ```
6. Start the Flask server:
   ```
   python run.py
   ```
7. In a separate terminal, start the Celery worker:
   ```
   celery -A app.tasks.celery_app worker --loglevel=info
   ```
8. Open frontend/index.html in your browser. The backend runs on http://localhost:5000.

10. Conclusion

The Placement Portal Application successfully addresses the real-world challenge of managing campus placements in a structured and efficient manner. By implementing a role-based access system for three distinct user types — students, companies, and administrators — the platform ensures that each stakeholder interacts only with the features relevant to them.

The modular Flask Blueprint architecture makes the codebase scalable and maintainable, while asynchronous background tasks via Celery and Redis enable automated communication workflows that would otherwise require manual effort. The Vue.js frontend provides a clean, reactive user experience through a single-page application that communicates with the backend via well-defined RESTful API endpoints.

This project demonstrates a practical implementation of full-stack web development principles, combining backend engineering, database design, API architecture, async task processing, and modern frontend development into a cohesive, production-ready application.